Recurrent Hidden State Dynamics, Backpropagation Through Time (BPTT), and Gated Architectures (LSTM & GRU)

A Unified Mathematical, Information-Theoretic, and Physical Foundation of Recurrent Neural Networks (RNN)

Contents

1 The Recurrent Neural Network Paradigm

Feedforward Neural Networks (MLPs) and Convolutional Neural Networks (CNNs) operate on fixed-dimensional spatial inputs under the assumption that observations are sampled independently and identically distributed (i.i.d.). However, sequential processes—such as natural language, time-series telemetry, and dynamical systems—exhibit variable sequence lengths and temporal auto-correlations.

Recurrent Neural Networks (RNNs) introduce cyclical internal connections, maintaining a persistent **Hidden State Vector** $\mathbf{h}_t \in \mathbb{R}^{d_h}$ that acts as an implicit continuous memory of all past observations $(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t)$.

Definition 1.1 (**Vanilla Recurrent Hidden State Dynamics**)**.** Let $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T)$ represent an input sequence where $\mathbf{x}_t \in \mathbb{R}^{d_x}$ at time step $t \in \{1, 2, \dots, T\}$. A Vanilla RNN updates its hidden state vector $\mathbf{h}_t \in \mathbb{R}^{d_h}$ and evaluates prediction vector $\hat{\mathbf{y}}_t \in \mathbb{R}^{d_y}$ via the non-linear recurrence equations:

$$\mathbf{a}_t = \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h \in \mathbb{R}^{d_h} \tag{1}$$

$$\mathbf{h}_t = \tanh(\mathbf{a}_t) \in \mathbb{R}^{d_h} \tag{2}$$

$$\mathbf{z}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y \in \mathbb{R}^{d_y} \tag{3}$$

$$\hat{\mathbf{y}}_t = \sigma_y(\mathbf{z}_t) \in \mathbb{R}^{d_y} \tag{4}$$

where:

- $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$ is the recurrent hidden-to-hidden transition matrix.
- $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times d_x}$ is the input-to-hidden projection matrix.
- $\mathbf{W}_{hy} \in \mathbb{R}^{d_y \times d_h}$ is the hidden-to-output emission matrix.
- $\mathbf{b}_h \in \mathbb{R}^{d_h}$ and $\mathbf{b}_y \in \mathbb{R}^{d_y}$ are bias vectors.
- $\mathbf{h}_0 \in \mathbb{R}^{d_h}$ is the initial hidden state vector (typically $\mathbf{h}_0 = \mathbf{0}$).
- σ_y is the task-specific emission activation (e.g., Softmax for discrete sequences).

Stationary parameter sharing ($\mathbf{W}_{hh}, \mathbf{W}_{xh}, \mathbf{W}_{hy}$ remaining constant across all time steps t) enables the model to process sequences of arbitrary length T while preserving temporal translation equivariance.

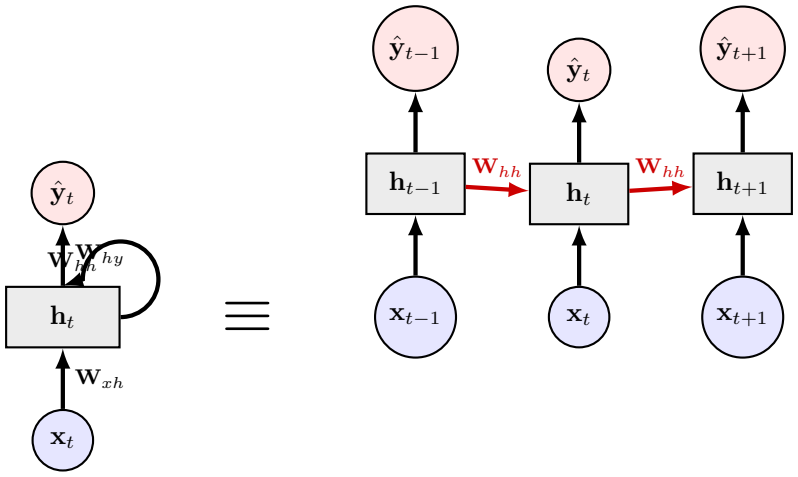

Figure 1: Folded graph representation vs. unrolled computational graph of a Recurrent Neural Network across sequence time.

2 Backpropagation Through Time (BPTT) and Gradient Dynamics

Training a Recurrent Neural Network over a sequence of length T involves unrolling its execution graph into a T-stage feedforward computational graph and calculating gradients using **Backpropagation Through Time (BPTT)**.

2.1 Analytical Derivation of BPTT Matrix Gradients

Let $\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t(\hat{\mathbf{y}}_t, \mathbf{y}_t)$ denote the total sequence loss.

Theorem 2.1 (**BPTT Gradient Expansions**)**.** The derivative of total sequence loss $\mathcal{L}$ with respect to the recurrent matrix $\mathbf{W}_{hh}$ accumulates temporal error contributions across all past time steps $k \leq t$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \sum_{k=1}^{t} \boldsymbol{\delta}_{t,k} \, \mathbf{h}_{k-1}^{T} \in \mathbb{R}^{d_h \times d_h} \tag{5}$$

where the temporal error propagation vector $\boldsymbol{\delta}_{t,k} \equiv \frac{\partial \mathcal{L}_t}{\partial \mathbf{a}_k} \in \mathbb{R}^{d_h}$ satisfies the exact Jacobian chain rule:

$$\boldsymbol{\delta}_{t,k} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{a}_k} = \left(\prod_{j=k+1}^{t} \frac{\partial \mathbf{a}_j}{\partial \mathbf{a}_{j-1}} \right) \frac{\partial \mathcal{L}_t}{\partial \mathbf{a}_t} \tag{6}$$

Proof. Evaluating the local step Jacobian matrix $\frac{\partial \mathbf{a}_j}{\partial \mathbf{a}_{j-1}}$:

$$\frac{\partial \mathbf{a}_j}{\partial \mathbf{a}_{j-1}} = \frac{\partial \mathbf{h}_{j-1}}{\partial \mathbf{a}_{j-1}} \frac{\partial \mathbf{a}_j}{\partial \mathbf{h}_{j-1}} = \operatorname{diag}\left(1 - \mathbf{h}_{j-1}^2\right) \mathbf{W}_{hh}^{T} \in \mathbb{R}^{d_h \times d_h} \tag{7}$$

Substituting this back into the product chain rule from time t back to time k:

$$\frac{\partial \mathbf{a}_t}{\partial \mathbf{a}_k} = \prod_{j=k+1}^{t} \left(\operatorname{diag}\left(1 - \mathbf{h}_{j-1}^2\right) \mathbf{W}_{hh}^{T} \right) \tag{8}$$

Accumulating over all predictions t and historical inputs $k \leq t$ completes the derivation. □

2.2 The Vanishing and Exploding Gradient Problem

The Jacobian product $\prod_{j=k+1}^{t} \frac{\partial \mathbf{a}_j}{\partial \mathbf{a}_{j-1}}$ governs long-range temporal gradient flow. Let $\gamma = \|\mathbf{W}_{hh}^{T}\|_2 \cdot \max_j \|\operatorname{diag}(1 - \mathbf{h}_{j-1}^2)\|_2$ be an upper bound on the spectral norm of the step Jacobian.

Taking the norm of the temporal error signal over $\tau = t - k$ time steps:

$$\left\| \frac{\partial \mathbf{a}_t}{\partial \mathbf{a}_k} \right\|_2 \leq \prod_{j=k+1}^{t} \left\| \frac{\partial \mathbf{a}_j}{\partial \mathbf{a}_{j-1}} \right\|_2 \leq \gamma^{t-k} = \gamma^{\tau} \tag{9}$$

- **Vanishing Gradients ($\gamma < 1$):** As temporal distance $\tau \to \infty$, $\gamma^{\tau} \to 0$ exponentially fast. The network loses the ability to learn long-range temporal dependencies, as gradients from distant time steps approach zero.
- **Exploding Gradients ($\gamma > 1$):** As $\tau \to \infty$, $\gamma^{\tau} \to \infty$ exponentially, causing numeric overflow ('NaN' instabilities).

Remark 2.1 (**Gradient Norm Clipping**)**.** To prevent exploding gradients, Pascanu et al. (2013) proposed **Gradient Norm Clipping**. If the global gradient norm exceeds threshold τ_{clip}, the gradient vector is rescaled:

$$\mathbf{g} \leftarrow \begin{cases} \tau_{\text{clip}} \frac{\mathbf{g}}{\|\mathbf{g}\|_2}, & \text{if } \|\mathbf{g}\|_2 > \tau_{\text{clip}} \\ \mathbf{g}, & \text{otherwise} \end{cases} \tag{10}$$

3 Gated Architectures: LSTM and GRU

To eliminate vanishing gradients without heuristic clipping, gated architectures introduce additive memory channels that maintain stable gradient paths over long time horizons.

3.1 Long Short-Term Memory (LSTM) Networks

Introduced by Hochreiter and Schmidhuber (1997), the **Long Short-Term Memory (LSTM)** architecture decouples internal linear memory tracking from output representation by introducing a continuous **Cell State Vector** $\mathbf{c}_t \in \mathbb{R}^{d_h}$ governed by three multiplicative gating units.

Definition 3.1 (**LSTM Gating and State Equations**). For input $\mathbf{x}_t \in \mathbb{R}^{d_x}$ and previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$, let $[\mathbf{h}_{t-1}, \mathbf{x}_t] \in \mathbb{R}^{d_h+d_x}$ denote concatenation. The LSTM update equations are:

$$\text{Forget Gate:} \quad \mathbf{f}_t = \sigma\left(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f\right) \in (0,1)^{d_h} \tag{11}$$

$$\text{Input Gate:} \quad \mathbf{i}_t = \sigma\left(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i\right) \in (0,1)^{d_h} \tag{12}$$

$$\text{Candidate Cell State:} \quad \tilde{\mathbf{c}}_t = \tanh\left(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c\right) \in (-1,1)^{d_h} \tag{13}$$

$$\text{Cell State Update:} \quad \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \in \mathbb{R}^{d_h} \tag{14}$$

$$\text{Output Gate:} \quad \mathbf{o}_t = \sigma\left(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o\right) \in (0,1)^{d_h} \tag{15}$$

$$\text{Hidden State Update:} \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh\left(\mathbf{c}_t\right) \in (-1,1)^{d_h} \tag{16}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the element-wise logistic sigmoid function, and $\odot$ is the Hadamard product.

Mathematical Mechanism Resolving Vanishing Gradients

Differentiating Equation (14) with respect to the previous cell state $\mathbf{c}_{t-1}$:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \operatorname{diag}(\mathbf{f}_t) + \mathbf{c}_{t-1}^T \frac{\partial \mathbf{f}_t}{\partial \mathbf{c}_{t-1}} + \tilde{\mathbf{c}}_t^T \frac{\partial \mathbf{i}_t}{\partial \mathbf{c}_{t-1}} + \mathbf{i}_t^T \frac{\partial \tilde{\mathbf{c}}_t}{\partial \mathbf{c}_{t-1}} \tag{17}$$

If the network learns to keep the forget gate open ($\mathbf{f}_t \approx \mathbf{1}$), the Jacobian term simplifies to:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx \mathbf{I} \implies \prod_{j=k+1}^{t} \frac{\partial \mathbf{c}_j}{\partial \mathbf{c}_{j-1}} \approx \mathbf{I} \tag{18}$$

This linear, additive cell state pathway creates a **Constant Error Carousel (CEC)**, allowing error signals to backpropagate over thousands of time steps without exponential decay.

3.2 Gated Recurrent Unit (GRU)

Cho et al. (2014) simplified the LSTM by merging the cell state and hidden state into a single vector $\mathbf{h}_t \in \mathbb{R}^{d_h}$, controlled by two gating units: a **Reset Gate** $\mathbf{r}_t$ and an **Update Gate** $\mathbf{z}_t$.

Definition 3.2 (**GRU Gating and State Equations**). For input $\mathbf{x}_t$ and previous hidden state $\mathbf{h}_{t-1}$:

$$\text{Reset Gate:} \quad \mathbf{r}_t = \sigma\left(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r\right) \in (0,1)^{d_h} \tag{19}$$

$$\text{Update Gate:} \quad \mathbf{z}_t = \sigma\left(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z\right) \in (0,1)^{d_h} \tag{20}$$

$$\text{Candidate Hidden State:} \quad \tilde{\mathbf{h}}_t = \tanh\left(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h\right) \in (-1,1)^{d_h} \tag{21}$$

$$\text{Hidden State Interpolation:} \quad \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \in \mathbb{R}^{d_h} \tag{22}$$

The update gate $\mathbf{z}_t$ acts as a linear interpolator between the historical hidden state $\mathbf{h}_{t-1}$ and the candidate state $\tilde{\mathbf{h}}_t$. If $\mathbf{z}_t \to \mathbf{0}$, the hidden state remains unchanged across time step t ($\mathbf{h}_t = \mathbf{h}_{t-1}$), maintaining long-range memory.

Feature Metric	Vanilla RNN	LSTM	GRU
State Vectors	$\mathbf{h}_t$ (Hidden)	$\mathbf{c}_t$ (Cell), $\mathbf{h}_t$ (Hidden)	$\mathbf{h}_t$ (Hidden)
Gating Units	None	3 ($\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t$)	2 ($\mathbf{r}_t, \mathbf{z}_t$)
Weight Matrices	2 ($\mathbf{W}_{hh}, \mathbf{W}_{xh}$)	4 ($\mathbf{W}_f, \mathbf{W}_i, \mathbf{W}_c, \mathbf{W}_o$)	3 ($\mathbf{W}_r, \mathbf{W}_z, \mathbf{W}_h$)
Parameter Count	$\mathcal{O}(d_h^2 + d_h d_x)$	$\mathcal{O}(4(d_h^2 + d_h d_x))$	$\mathcal{O}(3(d_h^2 + d_h d_x))$
Long-Range Flow	Vanishing/Exploding	Constant Error Carousel	Linear Interpolation

Table 1: Structural and parameter complexity comparison across recurrent architectures.

3.3 Architectural Comparison: Vanilla RNN vs. LSTM vs. GRU

4 Information-Theoretic and Physical Analogies

4.1 Information-Theoretic View: Non-Stationary Information Bottlenecks

In information theory, sequence processing can be framed through temporal **Predictive Rate-Distortion**.

A recurrent model constructs a continuous Markov chain $\mathbf{x}_1 \to \mathbf{x}_2 \to \cdots \to \mathbf{x}_T \to \mathbf{Y}$. The hidden state $\mathbf{h}_t$ acts as a continuous non-stationary **Information Bottleneck**, compressing past sequence history $X_{1:t}$ into a finite-capacity vector $\mathbf{h}_t$:

$$\min_{p(\mathbf{h}_t|X_{1:t})} \mathcal{L}_{\text{IB}} = I(X_{1:t}; \mathbf{h}_t) - \beta I(\mathbf{h}_t; \mathbf{y}_t) \tag{23}$$

LSTM/GRU gates adjust the local mutual information flow dynamically: * The forget gate $\mathbf{f}_t$ controls the rate at which historical entropy $I(X_{1:t-1}; \mathbf{c}_{t-1})$ is discarded. * The input gate $\mathbf{i}_t$ controls the insertion rate of novel innovation entropy $I(\mathbf{x}_t; \tilde{\mathbf{c}}_t)$.

4.2 Physical Analogy: Driven Non-Linear Dynamical Systems and Attractors

The hidden state update $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$ corresponds to a continuous-time **Driven Dissipative Dynamical System** discretized via Forward Euler integration:

$$\frac{d\mathbf{h}(t)}{dt} = -\frac{1}{\tau_{\text{sys}}}\mathbf{h}(t) + \mathbf{\Phi}\left(\mathbf{W}_{hh}\mathbf{h}(t) + \mathbf{W}_{xh}\mathbf{x}(t)\right) \tag{24}$$

* **Fixed Point Attractors:** Stationary sequence regimes pull hidden state trajectories $\mathbf{h}(t)$ into stable fixed-point basins in phase space $\mathbb{R}^{d_h}$. * **Bifurcations:** Input transitions $\mathbf{x}_t$ trigger phase bifurcations, shifting trajectories between distinct attractor basins. * **Gated Dynamics as Time-Constant Modulation:** In LSTMs, the forget gate $\mathbf{f}_t$ dynamically modulates the local system relaxation time constant $\tau_{\text{sys}}(\mathbf{x}_t)$, scaling fluid memory persistence across temporal scales.

5 Production-Grade Vectorized NumPy Implementation

The following Python code provides a complete, production-grade implementation of a Multi-Layer Long Short-Term Memory (LSTM) engine built strictly from scratch using vectorized NumPy operations, including unrolled temporal forward passes, full BPTT gradient updates, and Adam optimization.

```
import numpy as np

def sigmoid(x: np.ndarray) -> np.ndarray:
    """Element-wise stable logistic sigmoid function."""
    return 1.0 / (1.0 + np.exp(-np.clip(x, -15.0, 15.0)))


class LSTMLayerNumPy:
```

```
    """
    Production-grade Long Short-Term Memory (LSTM) Layer built strictly from scratch in NumPy.
    Handles variable sequence dimensions, unrolled forward state tracking, and full BPTT gradients.
    """
    def __init__(self, input_dim: int, hidden_dim: int):
        self.input_dim = input_dim
        self.hidden_dim = hidden_dim

        # Combined weight matrices for [f, i, c_tilde, o] gates: shape (4 * hidden_dim, hidden_dim + input_dim)
        concat_dim = hidden_dim + input_dim
        scale = np.sqrt(2.0 / concat_dim)
        self.W = np.random.randn(4 * hidden_dim, concat_dim) * scale
        self.b = np.zeros((4 * hidden_dim, 1))

        # Forget gate bias initialization trick (set to +1.0 for stable gradient flow)
        self.b[:hidden_dim] = 1.0

        # Parameter Gradients
        self.dW = None
        self.db = None

        # State Cache for Backpropagation Through Time (BPTT)
        self.x_cache = None
        self.h_cache = None
        self.c_cache = None
        self.gates_cache = None

    def forward(self, X: np.ndarray) -> np.ndarray:
        """
        Unrolled Forward Pass across sequence length T.
        Input X shape: (T, batch_size, input_dim)
        Output H shape: (T, batch_size, hidden_dim)
        """
        T, B, d_x = X.shape
        d_h = self.hidden_dim

        H = np.zeros((T, B, d_h))
        C = np.zeros((T, B, d_h))

        # Caches
        self.x_cache = X
        self.h_cache = np.zeros((T + 1, B, d_h))
        self.c_cache = np.zeros((T + 1, B, d_h))
        self.gates_cache = np.zeros((T, B, 4 * d_h))

        h_prev = np.zeros((B, d_h))
        c_prev = np.zeros((B, d_h))

        for t in range(T):
            x_t = X[t] # (B, d_x)

            # Concatenate hidden state and input: (B, d_h + d_x)
            concat = np.hstack((h_prev, x_t))

            # Linear projection: (B, 4 * d_h)
            gates_linear = concat @ self.W.T + self.b.T
```

```
65
66              # Split into gate components
67              f_gate = sigmoid(gates_linear[:, :d_h])
68              i_gate = sigmoid(gates_linear[:, d_h:2*d_h])
69              c_tilde = np.tanh(gates_linear[:, 2*d_h:3*d_h])
70              o_gate = sigmoid(gates_linear[:, 3*d_h:])
71
72              # Cell state update: c_t = f * c_{t-1} + i * c_tilde
73              c_t = f_gate * c_prev + i_gate * c_tilde
74
75              # Hidden state update: h_t = o * tanh(c_t)
76              h_t = o_gate * np.tanh(c_t)
77
78              # Store in sequence arrays
79              H[t] = h_t
80              C[t] = c_t
81
82              # Cache values for BPTT
83              self.h_cache[t] = h_prev
84              self.c_cache[t] = c_prev
85              self.gates_cache[t] = np.hstack((f_gate, i_gate, c_tilde,
    o_gate))
86
87              h_prev = h_t
88              c_prev = c_t
89
90          self.h_cache[T] = h_prev
91          self.c_cache[T] = c_prev
92          return H
93
94      def backward(self, dH: np.ndarray) -> np.ndarray:
95          """
96          Backpropagation Through Time (BPTT) evaluating parameter gradients
    dW, db
97          and propagating input gradient dX.
98          Input dH shape: (T, batch_size, hidden_dim)
99          Output dX shape: (T, batch_size, input_dim)
100         """
101         T, B, d_h = dH.shape
102         d_x = self.input_dim
103
104         self.dW = np.zeros_like(self.W)
105         self.db = np.zeros_like(self.b)
106         dX = np.zeros((T, B, d_x))
107
108         dh_next = np.zeros((B, d_h))
109         dc_next = np.zeros((B, d_h))
110
111         for t in reversed(range(T)):
112             x_t = self.x_cache[t]
113             h_prev = self.h_cache[t]
114             c_prev = self.c_cache[t]
115
116             f_gate = self.gates_cache[t, :, :d_h]
117             i_gate = self.gates_cache[t, :, d_h:2*d_h]
118             c_tilde = self.gates_cache[t, :, 2*d_h:3*d_h]
119             o_gate = self.gates_cache[t, :, 3*d_h:]
120
121             c_t = f_gate * c_prev + i_gate * c_tilde
122             tanh_c_t = np.tanh(c_t)
```

```
123
124             # Total incoming hidden gradient
125             dh = dH[t] + dh_next
126
127             # Output gate gradient
128             do = dh * tanh_c_t
129             do_linear = do * o_gate * (1.0 - o_gate)
130
131             # Cell state gradient (combines current h_t gradient and next
    step c_{t+1} gradient)
132             dc = dh * o_gate * (1.0 - tanh_c_t**2) + dc_next
133
134             # Forget gate gradient
135             df = dc * c_prev
136             df_linear = df * f_gate * (1.0 - f_gate)
137
138             # Input gate gradient
139             di = dc * c_tilde
140             di_linear = di * i_gate * (1.0 - i_gate)
141
142             # Candidate cell state gradient
143             dc_tilde = dc * i_gate
144             dc_tilde_linear = dc_tilde * (1.0 - c_tilde**2)
145
146             # Concatenate gate gradients: (B, 4 * d_h)
147             d_gates_linear = np.hstack((df_linear, di_linear,
    dc_tilde_linear, do_linear))
148
149             # Concatenated input vector: (B, d_h + d_x)
150             concat = np.hstack((h_prev, x_t))
151
152             # Accumulate weight and bias gradients
153             self.dW += d_gates_linear.T @ concat / B
154             self.db += np.sum(d_gates_linear, axis=0, keepdims=True).T / B
155
156             # Propagate gradient to concatenated input
157             d_concat = d_gates_linear @ self.W
158             dh_next = d_concat[:, :d_h]
159             dX[t] = d_concat[:, d_h:]
160
161             # Propagate cell gradient to previous step
162             dc_next = dc * f_gate
163
164         return dX
165
166
167 if __name__ == "__main__":
168     np.random.seed(42)
169
170     # Generate Synthetic Sequence Batch (T=10 time steps, B=4 batch size,
    d_x=5 features)
171     T_steps, B_batch, d_in = 10, 4, 5
172     d_hidden = 8
173
174     X_seq = np.random.randn(T_steps, B_batch, d_in)
175
176     # Initialize LSTM Layer
177     lstm = LSTMLayerNumPy(input_dim=d_in, hidden_dim=d_hidden)
178
179     # Unrolled Forward Pass
```

```
    H_out = lstm.forward(X_seq)

    print("=== RECURRENT LSTM ENGINE INITIALIZATION SUCCESSFUL ===")
    print(f"Sequence Length (T) : {T_steps}")
    print(f"Batch Size (B)      : {B_batch}")
    print(f"Input Features (d_x): {d_in}")
    print(f"Hidden Dim (d_h)    : {d_hidden}")
    print(f"Output Hidden Shape : {H_out.shape} (T, B, d_h)")

    # Execute Backward Pass (BPTT)
    dH_dummy = np.random.randn(*H_out.shape)
    dX_grad = lstm.backward(dH_dummy)

    print(f"BPTT Input Gradient Shape: {dX_grad.shape}")
    print(f"Accumulated dW Weight Matrix Shape: {lstm.dW.shape}")
```

Listing 1: Vectorized NumPy implementation of a complete LSTM Layer with BPTT.